

NAME: SHRI SANJANAA M

ROLL NO.:24BCS265

## Scenario: Public Transport Ticket Booking and Fare Processing System

A public transport authority is developing a Java application to manage **ticket booking and fare calculation** for passengers.

All ticket-related information must be stored **inside a single class using private data members**.

Ticket details must be **read and modified only through public methods**.

During execution, ticket details are entered at runtime.

Some tickets are created using **only basic passenger information**, and complete travel information is added later during booking confirmation.

---

### Exercise Requirements

#### Step 1: Runtime Ticket Input

Accept the following values from the user during program execution:

- Ticket Number (integer)
- Passenger Name (string)
- Passenger Age (integer)
- Travel Distance in kilometers (integer)
- Travel Class (string: "General", "Sleeper", "AC")

Store each value in a **private data member** and display all values using a public method.

---

#### Step 2: Object Creation Using Limited and Complete Data

Create:

- One ticket object using **only**:
  - Ticket Number
  - Passenger Name
  - Passenger Age
- One ticket object using **all five values** listed in Step 1

Display both ticket objects using public methods.

---

#### Step 3: Reference Assignment and Data Update

Assign the first ticket object created in Step 2 to a second reference variable.

Using the second reference variable:

- Update the travel distance to a new value using a public method

Display ticket details using **both reference variables**.

---

#### Step 4: Independent Object Creation

Create a **new ticket object** by passing values obtained from an existing ticket object.

Using a public method:

- Change the travel class of the new ticket object to "AC"

Display details of both ticket objects.

---

#### Step 5: Update Method with Identical Parameter Names

Create a public method to update ticket details where:

- The method parameters are named **exactly the same** as the class data members
- The correct ticket data members are updated inside the method

Display the updated ticket details.

---

#### Step 6: Fare Calculation Without Conditional Statements

Create a public method that accepts a ticket object and calculates fare using the following **explicit rules**:

- Base fare per kilometer = 10
- Travel class multiplier:
  - "General" → multiplier 1
  - "Sleeper" → multiplier 2
  - "AC" → multiplier 3
- Total Fare = travel distance × base fare × multiplier

The fare calculation **must not use if, if-else, or ternary operators**.

Display the calculated fare using a public method.

---

#### Step 7: Data Access Control

Ensure that:

- All data members are declared private
  - No data member is accessed directly outside the class
  - All read and update operations are performed through public methods only
- 

#### Step 8: Temporary Object Creation

Within the fare-calculation process:

- Create at least one ticket object used only for fare comparison
- Do not store this object in any class-level variable

- After fare comparison, continue execution using only the main ticket object
- 

## Inference Questions

1. **(Step 1)** How does accepting ticket details at runtime allow the same program to handle different passengers without changing the source code?

Accepting ticket details at runtime allows the program to take **different passenger inputs each time it runs**. The same code can handle multiple passengers without modifying the source code.

2. **(Step 2)** In what way does creating a ticket with limited details restrict or delay certain operations compared to a fully initialized ticket?

A ticket created with limited details does not contain travel distance or travel class. Therefore, operations like **fare calculation or booking confirmation cannot be performed until the missing data is added**.

3. **(Step 3)** If two different modules update the same ticket through shared references, what risks or benefits arise in maintaining ticket data?

When two references point to the same ticket object, **changes made using one reference affect the other**. This saves memory but may cause **data modification risks if multiple modules update it**.

4. **(Step 4)** Why is creating a separate ticket object safer than reusing the same object when testing changes such as travel class upgrades?

Creating a new ticket object keeps the **original ticket unchanged**. This allows safe testing of changes like travel class upgrades without affecting the original booking data.

5. **(Step 5)** What kind of incorrect behavior would occur if the program failed to distinguish between method parameters and class data members?

If method parameters and class variables have the same names without proper distinction (using this), the **class data members may not update correctly**, causing incorrect ticket information.

6. **(Step 6)** How does using a rule-based calculation instead of conditional statements improve or limit future changes in fare policy?

Using rule-based fare calculation avoids complex conditional statements and makes the system **easier to maintain and update** if fare rules change.

7. **(Step 7)** If ticket data members were publicly accessible, how could this affect fare accuracy and system reliability?

If ticket data members were public, any part of the program could modify them directly. This could lead to **incorrect ticket data and unreliable fare calculations**.

8. **(Step 8)** Why is it important to restrict the lifetime of ticket objects that are created only for comparison or testing?

Temporary ticket objects are used only for short tasks like comparison. Restricting their lifetime **saves memory and prevents confusion with actual ticket data**.

**Code:**

```
import java.util.Scanner;

import java.util.HashMap;
```

```
class Ticket {
```

```
    // Step 7: Private data members
```

```
    private int ticketNumber;
```

```
    private String passengerName;
```

```
    private int passengerAge;
```

```
    private int travelDistance;
```

```
    private String travelClass;
```

```
    // Constructor with limited details
```

```
    public Ticket(int ticketNumber, String passengerName, int passengerAge) {
```

```
        this.ticketNumber = ticketNumber;
```

```
        this.passengerName = passengerName;
```

```
        this.passengerAge = passengerAge;
```

```
    }
```

```
    // Constructor with all details
```

```
    public Ticket(int ticketNumber, String passengerName, int passengerAge,
```

```
        int travelDistance, String travelClass) {
```

```
        this.ticketNumber = ticketNumber;
```

```
        this.passengerName = passengerName;
```

```
        this.passengerAge = passengerAge;
```

```
        this.travelDistance = travelDistance;
```

```
        this.travelClass = travelClass;
```

```
    }
```

```
    // Copy constructor (Step 4)
```

```
    public Ticket(Ticket t) {
```

```
        this.ticketNumber = t.ticketNumber;
```

```
        this.passengerName = t.passengerName;
```

```
        this.passengerAge = t.passengerAge;
```

```
        this.travelDistance = t.travelDistance;
```

```

        this.travelClass = t.travelClass;
    }

    // Display method
    public void displayTicket() {
        System.out.println("Ticket Number: " + ticketNumber);
        System.out.println("Passenger Name: " + passengerName);
        System.out.println("Passenger Age: " + passengerAge);
        System.out.println("Travel Distance: " + travelDistance);
        System.out.println("Travel Class: " + travelClass);
        System.out.println("-----");
    }

    // Update distance
    public void setTravelDistance(int travelDistance) {
        this.travelDistance = travelDistance;
    }

    // Update travel class
    public void setTravelClass(String travelClass) {
        this.travelClass = travelClass;
    }

    // Step 5: Update method with identical parameter names
    public void updateTicket(int ticketNumber, String passengerName, int passengerAge,
                             int travelDistance, String travelClass) {

        this.ticketNumber = ticketNumber;
        this.passengerName = passengerName;
        this.passengerAge = passengerAge;
        this.travelDistance = travelDistance;
        this.travelClass = travelClass;
    }

    // Fare calculation without if/else

```

```

public static void calculateFare(Ticket t) {
    int baseFare = 10;
    HashMap<String, Integer> map = new HashMap<>();
    map.put("General", 1);
    map.put("Sleeper", 2);
    map.put("AC", 3);
    int multiplier = map.get(t.travelClass);
    int fare = t.travelDistance * baseFare * multiplier;
    // Step 8: Temporary object
    Ticket temp = new Ticket(0, "Temp", 0, 10, "General");
    int tempFare = temp.travelDistance * baseFare * map.get(temp.travelClass);
    System.out.println("Calculated Fare: " + fare);
    if (fare > tempFare) {
        System.out.println("Fare is higher than temporary ticket fare.");
    }
}
}
}

```

```

public class MainTicketSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Step 1: Runtime Input
        System.out.println("Enter Ticket Number:");
        int tno = sc.nextInt();
        sc.nextLine();
        System.out.println("Enter Passenger Name:");
        String name = sc.nextLine();
        System.out.println("Enter Passenger Age:");
        int age = sc.nextInt();
        System.out.println("Enter Travel Distance:");
        int distance = sc.nextInt();
        sc.nextLine();
        System.out.println("Enter Travel Class (General/Sleeper/AC):");
        String tclass = sc.nextLine();
        Ticket runtimeTicket = new Ticket(tno, name, age, distance, tclass);
    }
}

```

```

System.out.println("\nRuntime Ticket Details");
runtimeTicket.displayTicket();

// Step 2: Object creation
Ticket ticket1 = new Ticket(101, "Ravi", 25);
Ticket ticket2 = new Ticket(102, "Priya", 30, 150, "Sleeper");
System.out.println("Ticket with Limited Details");
ticket1.displayTicket();

System.out.println("Ticket with Complete Details");
ticket2.displayTicket();

// Step 3: Reference assignment
Ticket refTicket = ticket1;
refTicket.setTravelDistance(200);
System.out.println("After Updating Distance Using Second Reference");
ticket1.displayTicket();
refTicket.displayTicket();

// Step 4: Independent object
Ticket newTicket = new Ticket(ticket2);
newTicket.setTravelClass("AC");
System.out.println("Original Ticket");
ticket2.displayTicket();
System.out.println("New Modified Ticket");
newTicket.displayTicket();

// Step 5: Update method
ticket1.updateTicket(201, "Kumar", 40, 300, "General");
System.out.println("Updated Ticket Details");
ticket1.displayTicket();

// Step 6: Fare Calculation
Ticket.calculateFare(runtimeTicket);
sc.close();
}
}

```

**OUTPUT:**



<terminated> sanju [Java Application] C:\Users\Shri Sanjanaa\.p2\pc

Enter Ticket Number:

7

Enter Passenger Name:

Sanjana

Enter Passenger Age:

19

Enter Travel Distance:

70

Enter Travel Class (General/Sleeper/AC):

ac

Runtime Ticket Details

Ticket Number: 7

Passenger Name: Sanjana

Passenger Age: 19

Travel Distance: 70

Travel Class: ac

-----

Ticket with Limited Details

Ticket Number: 101

Passenger Name: Ravi

Passenger Age: 25

Travel Distance: 0

Travel Class: null

-----

Ticket with Complete Details

Ticket Number: 102

Passenger Name: Priya

Passenger Age: 30

Travel Distance: 150

Travel Class: Sleeper

-----

---

# After Updating Distance Using Second Reference

Ticket Number: 101

Passenger Name: Ravi

Passenger Age: 25

Travel Distance: 200

Travel Class: null

-----  
Ticket Number: 101

Passenger Name: Ravi

Passenger Age: 25

Travel Distance: 200

Travel Class: null

-----  
Original Ticket

Ticket Number: 102

Passenger Name: Priya

Passenger Age: 30

Travel Distance: 150

Travel Class: Sleeper

-----  
New Modified Ticket

Ticket Number: 102

Passenger Name: Priya

Passenger Age: 30

Travel Distance: 150

Travel Class: AC

-----  
Updated Ticket Details

Ticket Number: 201

Passenger Name: Kumar

Passenger Age: 40

Travel Distance: 300

Travel Class: General

-----